

CollabBoard

Gerçek zamanlı işbirlikçi Kanban panosu — arayüz tasarım brief'i ve proje envanteri

CollabBoard — Arayüz Tasarım Brief'i

Bu belge ne için? Mevcut backend'in tamamını ve arayüzün karşılaması gereken her ihtiyacı tek yerde toplar. Bir tasarımcıya (veya tasarım yapan bir yapay zekâya) verildiğinde, projeyi hiç görmemiş biri bile eksiksiz bir arayüz tasarlayabilmelidir.

1. Ürün tek cümlede

CollabBoard, birden çok kişinin **aynı anda** düzenlediği, her değişikliğin herkeste **anında** görüldüğü bir Kanban panosudur (Trello benzeri). Bir kişi kartı taşıdığında diğerlerinin ekranında da kayar — kimse sayfayı yenilemez.

Farkı ne? Sıradan bir yapılacaklar uygulaması değil; **gerçek zamanlı işbirliği** ürünü. Tasarımın öne çıkarması gereken üç şey:

- Canlılık** — başkalarının yaptıkları anında görünür
- Farkındalık** — kim çevrimiçi, kim ne yaptı
- Güven** — çakışma olduğunda kullanıcı ne olduğunu anlar

2. Hiyerarşi ve veri modeli

Çalışma Alanı (Workspace / şirket)

└ Pano (Board)

└ Kolon (Column) ör. "Yapılacaklar", "Devam Ediyor", "Tamam"

└ Kart (Card) ör. "Süt al"

Nesne	Alanları	Notlar
Workspace	id , name , slug	Şirket/ekip. Her kullanıcının bir "Kişisel Alan"ı vardır.
Board	id , name , workspaceId , columns[]	Bir çalışma alanına aittir.
Column	id , name , position , cards[]	Soldan sağa sıralanır; sürüklenerek yeri değişir.
Card	id , title , position , version	version , çakışma kontrolü içindir (arayüzde gösterilmez).
User	id , firstName , lastName , email	
Activity	actorName , type , description , createdAt	Pano geçmişi.

3. Roller ve yetkiler ⚠ tasarımı doğrudan etkiler

İki seviye vardır. **Pano seviyesindeki rol, çalışma alanı rolünü ezer** (istisna).

Çalışma alanı rolleri

Rol	Şirketin panolarına	Üye yönetimi	Pano açma
OWNER (Yönetici)	tam yetki	✓	✓
ADMIN (Yönetici)	tam yetki	✓	✓
MEMBER (Üye)	düzenler	—	✓
GUEST (Misafir)	erişemez	—	—

Pano rolleri

Rol	Görür	Kart/kolon değiştirir	Üye yönetir
OWNER (Yönetici)	✓	✓	✓
EDITOR (Editör)	✓	✓	—
VIEWER (İzleyici)	✓	—	—

Tasarıma etkisi: **VIEWER** için kart ekleme alanı, silme ikonu ve sürükleme **gizlenmeli/pasif** olmalı; **OWNER** dışındakiler davet satırını görmemeli. Kullanıcı kendi rolünü her zaman görebilmeli (rozet).

Not: Arayüzdeki kısıtlamalar yalnızca deneyim içindir; yetkiyi sunucu uygular.

4. Tasarlanması gereken ekranlar

4.1 Giriş / Kayıt

- Alanlar: **Giriş** → e-posta, şifre · **Kayıt** → ad, soyad, e-posta, şifre (min. 8 karakter)
- İki mod arasında geçiş bağlantısı
- Hata durumu: "E-posta veya şifre hatalı", alan bazlı doğrulama hataları
- Marka anı: ürünün ne olduğunu anlatan kısa bir cümle burada olabilir

4.2 Çalışma alanı + pano seçimi

- Kullanıcının **çalışma alanları** ve her alandaki **panolar**
- Her pano satırında kullanıcının **rol rozeti**
- Yeni pano** ve **yeni çalışma alanı** oluşturma
- Boş durum:** hiç pano yokken yönlendirici bir ekran gerekir

4.3 Pano ekranı (ana ekran)

Ekranın kalbi. İçermesi gerekenler:

- Kolonlar** yan yana, yatay kaydırmalı; her kolonda başlık + kart sayısı
- Kartlar** dikey liste; sürük-le-bırak (kolon içinde ve kolonlar arası)
- Kolonlar da sürüklenebilir** (yeniden sıralama)
- Kart üzerinde: **silme** ikonu (üzerine gelince), **çift tıkla düzenle**
- Her kolonun altında **kart ekleme** alanı
- Üst bar: pano adı, **çevrimiçi kullanıcı avaturları**, **bağlantı durumu göstergesi**, çıkış
- Sağ panel: **Üyeler** ve **Son hareketler**
- Alt köşe: **canlı sistem göstergeleri** (isteğe bağlı, teknik vitrin)

4.4 Üyeler paneli

- Üye listesi: avatar (baş harfler), ad, e-posta, **rol rozeti**
- OWNER** için: e-posta ile **davet** (rol seçimli) ve **çıkarma**
- "(sen)" işareti kendi satırında

4.5 Son hareketler paneli (pano geçmişi)

- Kim, ne yaptı, ne zaman: "Erdem Sidal — 'Süt al' kartını Done kolonuna taşıdı — 2 dk önce"
- Renkli avatar; görelili zaman
- Canlı güncellenir (her olayda yenilenir)
- Boş durum** gerekir

4.6 Durum ve geri bildirim öğeleri

- Bağlantı göstergesi:** bağlı / bağlantı kesildi (renkli nokta + metin)

- **Bildirim (toast):** çakışma reddi, yeniden bağlanma, davet sonucu
- **Çakışma anı:** "Bu kartı senden önce biri değiştirdi. Ekran güncellendi, tekrar dene."

5. Gerçek zamanlı davranışlar (tasarımda hissettirilmeli)

Olay	Kullanıcı ne görmeli?
Başkası kart ekledi	Kart kolonda belirir (yumuşak giriş animasyonu)
Başkası kart taşıdı	Kart yeni yerine kayar
Başkası kartı düzenledi	Başlık değişir (kısa vurgulama düşünülebilir)
Başkası kartı sildi	Kart kaybolur
Biri panoya katıldı/ayrıldı	Avatar listesi anında güncellenir
Kendi işlemim reddedildi	Bildirim + ekran sunucudaki gerçeğe göre tazelenir
Bağlantı koptu	Gösterge kırmızıya döner; geri gelince bildirim + tazeleme

Önemli: Kullanıcı kendi hareketini **anında** görür (iyimser arayüz); sunucu reddederse geri alınır.

6. Mevcut arayüz (yeniden tasarlanacak olan)

Şu an çalışan, sade bir arayüz var. Tasarımın bunu **geliştirmesi** bekleniyor.

- **Renkler:** açık zemin `#eef3f8`, kart/panel beyaz, kenarlık `#e1e7ef`, metin `#1f2d3d`, ikincil metin `#6b7a90`
- **Vurgu rengi:** turuncu `#f5871f` (yumuşak tonu `#fff1e2`) — **korunması isteniyor**
- **Rol renkleri:** Yönetici turuncu, Editör mavi, İzleyici gri
- **Durum renkleri:** bağlı `#35b37e`, kopuk `#e05b49`
- **Yazı tipi:** sistem yazı tipi yığını
- **Yerleşim:** üst bar + yatay kaydırmalı kolonlar + sağda iki panel

Eksik/zayıf bulunan yönler (tasarımın çözmesi istenenler): - Çalışma alanı kavramı arayüzde henüz görünmüyor - Boş durumlar zayıf - Mobil/dar ekran düşünülmemiş - Kart detayı yok (şu an yalnızca başlık; `prompt()` ile düzenleniyor) - Görsel hiyerarşi ve nefes alanı geliştirilebilir

7. Teknik kısıtlar tasarımın uyması gereken sınırlar

- **Framework yok.** Arayüz tek bir `index.html` içinde: vanilya JavaScript + CSS. React/Vue/Tailwind kullanılmayacak.

- Dış kütüphaneler yalnızca CDN üzerinden ve sınırlı: **SortableJS** (sürükle-bırak), **@stomp/stompjs** (gerçek zamanlı bağlantı).
- İkonlar **satır içi SVG** olmalı (ikon kütüphanesi yok).
- Arayüz dili **Türkçe**.
- Tarayıcı hedefi: modern masaüstü tarayıcılar; dar ekranda kullanılabilir olması artı değer.
- Tasarım **uygulanabilir** olmalı: tek dosyaya sığacak, makul karmaşıklıkta CSS.

8. Backend arayüzü (API) — tasarımın dayandığı gerçek

Kimlik

Uç	Ne yapar
POST /api/auth/register	Kayıt (ad, soyad, e-posta, şifre)
POST /api/auth/login	Giriş → accessToken + refreshToken + kullanıcı bilgisi
POST /api/auth/refresh	Token yenileme (arka planda, kullanıcı görmez)

Panolar

Uç	Ne yapar
GET /api/boards	Erişebildiğim panolar (+ her birinde kendi rolüm)
POST /api/boards	Pano oluştur (name , isteğe bağlı workspaceId)
GET /api/boards/{id}	Panonun tam hâli: kolonlar + kartlar
GET /api/boards/{id}/activity	Pano geçmişi
GET/POST/DELETE /api/boards/{id}/members	Üye listesi / davet / çıkarma

Gerçek zamanlı kanal (WebSocket + STOMP)

Adres	Yön	İçerik
/topic/board.{id}	sunucu → herkes	Kart/kolon olayları
/topic/board.{id}/presence	sunucu → herkes	Çevrimiçi kullanıcı listesi
/user/queue/errors	sunucu → tek kişi	Reddedilen işlem bildirimi
/app/board/{id}/ops	istemci → sunucu	İşlem gönderimi

İşlem tipleri: ADD_CARD , MOVE_CARD , EDIT_CARD , DELETE_CARD , MOVE_COLUMN

9. Tasarımdan beklenen çıktı

1. **Ekran tasarımları:** giriş/kayıt · çalışma alanı & pano seçimi · pano ekranı · üyeler paneli · geçmiş paneli
2. **Durumlar:** boş · yükleniyor · hata · yetkisiz · bağlantı kopuk · çakışma bildirimi
3. **Bileşenler:** kart · kolon · avatar · rol rozeti · bildirim · davet satırı · buton/giriş alanı
4. **Etkileşim notları:** sürükleme sırasında görünüm, canlı güncelleme animasyonları
5. **Renk ve tipografi ölçeği** (mevcut turuncu vurgu korunarak)

10. Ürünün ruhu

Bu bir **ekip aracı**: sakın, hızlı ve güven veren olmalı. Kullanıcı ekrana baktığında şunu hissetmeli:

"Burada başkaları da var, ne yaptıkları belli, ve benim yaptığım anında herkese ulaşıyor."

Gösterişli olmasına gerek yok; **canlılık ve netlik** her şeyden önemli.

EK — Projede neler yapıldı (teknik envanter)

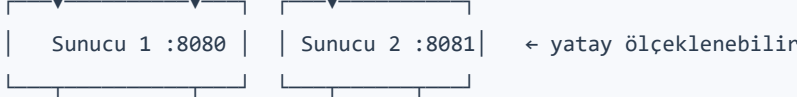
Tasarımcı için bağlam, geliştirici için hatırlatma. Arayüzün arkasında **çalışan** her şey burada.

Teknoloji yığını

Java 21 · Spring Boot 3 · WebSocket/STOMP · PostgreSQL · Redis · Docker · Flyway · JWT · Micrometer · Testcontainers
Frontend: vanilya JavaScript (tek dosya) + SortableJS + @stomp/stompjs

Mimari (alıřan hâli)

Tarayıcılar



REDIS (Pub/Sub + presence)

POSTGRESQL (panolar • kartlar • geçmiş • üyelikler)

İki kanal: Panoya giren istemci önce REST ile **tam fotoğrafı** alır, sonra WebSocket ile **o andan sonraki** değışiklikleri dinler.

Yapılan işler — fazlar

Faz	Ne yapıldı
1. Temel	Domain (pano/kolon/kart), REST uçları, WebSocket+STOMP altyapısı, operasyon tabanlı senkron, ilk arayüz
Cıla	Kart düzenleme/silme, kolon taşıma, SortableJS ile pürüzsüz sürükleme, görsel yenileme
2. Eşzamanlılık	Çakışma çözümü (sürüm kontrolü + reddetme + tazeleme), presence (kim çevrimiçi)
3. Ölçekleme	Redis Pub/Sub köprüsü — birden çok sunucu tek canlı yayın gibi çalışıyor
4. Olgunluk	JWT kimlik doğrulama (WebSocket dahil), pano geçmişı (audit), metrikler, README + mimari diyagramlar
5. Eriřim	Pano rolleri (Yönetici/Editör/İzleyici), üye yönetimi
6. Çalışma alanı	Şirket katmanı: panolar çalışma alanına ait, rol devralma

Çözölen zor problemler

- Çakışma:** İki kiři aynı kartı aynı anda düzenlerse, eski sürümle gelen **reddedilir**; kimsenin emeđi sessizce kaybolmaz. Reddetme yalnızca gönderene gider.
- Çok sunucu:** Bir sunucudaki değışiklik, diğeriine bađlı kullanıcılara ulaşmıyordu. Redis üzerinden yayın ile çözüldü.

- **WebSocket kimlik doğrulama:** Tarayıcı el sıkışmasına özel başlık ekleyemediği için token, STOMP `CONNECT` çerçevesinde taşınıyor (URL'de değil — log sızıntısı olmasın).
- **Okuma sızıntısı:** Üye olmayan biri panonun yayınını dinleyebiliyordu; abonelikler ayrıca denetleniyor.
- **Oturum sürekliliği:** Access token 15 dakikada dolarken kullanıcı atılıyordu; sessiz yenileme eklendi.

Kalite

- **23 otomatik test** (Testcontainers ile gerçek Postgres + Redis): canlı senkron, çıkışma, presence, roller, çalışma alanı devralma
- **6 mimari karar kaydı (ADR):** her önemli kararın gerekçesi ve reddedilen alternatifleri yazılı
- **7 veritabanı migration'ı** (Flyway), sıralı veri geçişleriyle

Bilinen eksikler (arayüz tasarımında fırsat)

- Çalışma alanı arayüzde henüz görünmüyor (REST hazır)
- Kart detayı yok — başlık `prompt()` ile düzenleniyor
- Mobil/dar ekran düşünülmemiş
- Boş durumlar zayıf
- İmleç paylaşımı (kimin faresi nerede) yok